



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Ingegneria
“Enzo Ferrari”

Fondamenti di elettronica

Esperienza di laboratorio xx: Comunicazione tramite BLE

Dott. Tommaso Zanotti

email: tommaso.zanotti@unimore.it

Breve introduzione al protocollo BLE

Bluetooth Low Energy (BLE) è una variante del Bluetooth tradizionale progettata per ridurre il consumo energetico, mantenendo comunque la capacità di trasferire dati a corto raggio. La sua principale applicazione è in dispositivi IoT, wearable e sensori, dove l'efficienza energetica è cruciale.

Basso Consumo

- BLE utilizza meno energia rispetto al Bluetooth classico, permettendo ai dispositivi di funzionare per lungo tempo senza ricaricare la batteria.

Ampia Portata

- Grazie alla sua portata elevata, BLE può connettere dispositivi a distanze maggiori rispetto al Bluetooth convenzionale.

Connessioni Rapide

- Il processo di accoppiamento e connessione tra dispositivi BLE è molto più veloce rispetto al Bluetooth classico.

Breve introduzione al protocollo BLE

Connessione Asincrona

- BLE utilizza una connessione asincrona che permette ai dispositivi di comunicare senza essere costantemente in contatto, risparmiando così energia

Architettura Master-Slave

- Il dispositivo master controlla e gestisce la connessione con i dispositivi slave, ottimizzando il consumo energetico

Multiplexing Adattivo

- BLE alterna tra diversi canali di comunicazione in modo dinamico per evitare interferenze e garantire una connessione stabile

Breve introduzione al protocollo BLE

Scenari di utilizzo

Domotica

Controllo e automazione di dispositivi per la casa, come luci, termostati e sensori di sicurezza.

Salute e Benessere

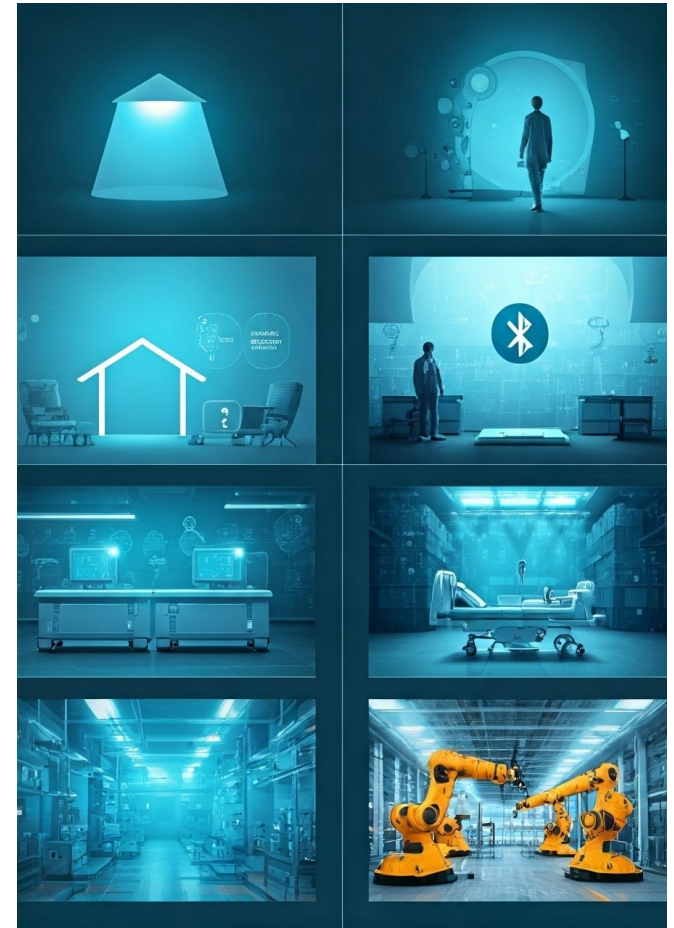
Monitoraggio della salute e del fitness tramite wearable e sensori BLE.

Industria 4.0

Connettività e monitoraggio di macchinari e asset industriali.

Automotive

Chiavi digitali, gestione dei veicoli e servizi di infotainment connessi.



Lo stack BLE



Il protocollo BLE è diviso in più livelli di astrazione chiamati *stack*, dove ogni livello ha una funzione specifica.

Physical Layer (PHY)

- Il livello fisico si occupa della trasmissione dei segnali radio. BLE usa la banda di frequenza dei 2.4 GHz e supporta trasmissioni a diverse velocità. La banda 2.4 GHz (ovvero la banda ISM (Industrial, Scientific, and Medical) non licenziata) è la stessa utilizzata dal Wi-Fi, microonde e altri dispositivi.
- Per ridurre le interferenze e ottimizzare l'uso della banda, BLE divide lo spettro in **40 canali fisici**, ciascuno largo **2 MHz**.
- Questi canali sono numerati da 0 a 39, e sono suddivisi in due categorie principali:
 - **Canali di advertising (3 canali)**: Canali 37, 38 e 39.
 - **Canali di dati (37 canali)**: Canali da 0 a 36.

Lo stack BLE

Physical Layer (PHY)

- **Canali di Advertising**
 - I canali di advertising sono usati per stabilire connessioni e per trasmettere pacchetti broadcast. Quando un dispositivo BLE vuole essere scoperto, trasmette pacchetti di advertising su questi tre canali. L'uso di più canali di advertising riduce la probabilità di interferenze e migliora la probabilità di essere scoperti.
- **Canali di Dati**
 - Dopo che una connessione BLE è stata stabilita tramite i canali di advertising, i dispositivi si spostano ai canali di dati. Sono disponibili 37 canali di dati (da 0 a 36) per trasferire effettivamente i dati tra i dispositivi. Per garantire un uso efficiente dello spettro e ridurre le interferenze con altri dispositivi che operano sulla stessa banda, BLE utilizza un meccanismo di salto di frequenza (frequency hopping).

Lo stack BLE



Link layer / Host controller interface

- Il livello link gestisce la connessione tra i dispositivi BLE e include funzioni come advertising, connessione, scansione e gestione delle connessioni. È qui che il dispositivo decide quando può inviare o ricevere dati.
- Uno sviluppatore può configurare il tipo di advertising (frequenza, potenza, intervallo) per ottimizzare la durata della batteria o migliorare la portata. Inoltre, può decidere se il dispositivo sarà un advertiser o un scanner.

Lo stack BLE



L2CAP (Logical Link Control and Adaptation Protocol)

Questo livello si occupa di dividere i dati in pacchetti e gestire il multiplexing tra canali logici.

Attribute Protocol (ATT)

L'**ATT** è il protocollo base che specifica come i dati (attributi) vengono trasmessi tra due dispositivi BLE. Ogni dispositivo BLE ha una tabella di attributi, e l'ATT definisce come leggere, scrivere o notificare questi attributi.

Lo stack BLE

Attribute Protocol (ATT)

Gli elementi chiave di ATT includono:

- **Attributi (Attributes):** Sono entità fondamentali e rappresentano i dati che devono essere scambiati tra dispositivi. Un attributo può essere un servizio, una caratteristica o un descrittore.
- **UUID:** Ogni attributo ha un identificatore univoco che può essere di 16 bit (standard) o 128 bit (custom).
- **Handle:** Un attributo è identificato anche da un numero chiamato *handle*. Questo *handle* è usato per accedere agli attributi, in modo simile a come gli indirizzi di memoria sono usati per accedere a dati specifici in una struttura di memoria.
- **Operazioni ATT:** ATT supporta operazioni come *Read*, *Write*, *Notify*, e *Indicate* per permettere ai dispositivi di scambiarsi informazioni sugli attributi.

L'ATT è un protocollo molto semplice che descrive come inviare pacchetti dati (attributi) tra due dispositivi BLE. Funziona in modalità **client-server**:

- **ATT Server:** Memorizza e gestisce la tabella degli attributi.
- **ATT Client:** Invia richieste per leggere o scrivere attributi nel server.

Esempio di ATT

Immagina che tu voglia leggere la temperatura da un dispositivo BLE. Il client invia una richiesta di lettura per un attributo specifico (ad esempio, la temperatura) utilizzando il suo handle. Il server risponde con il valore associato a quell'attributo.

Lo stack BLE



Generic Attribute Profile (GATT)

è uno dei livelli più importanti nello stack BLE perché definisce come i dati vengono organizzati, scambiati e strutturati tra i dispositivi BLE. Il livello GATT è dove si definiscono le interazioni tra il dispositivo periferico e quello centrale, attraverso una serie di servizi e caratteristiche.

GATT organizza i dati in una gerarchia ben definita che consente ai dispositivi di scambiarsi informazioni in modo standardizzato. Questa gerarchia è formata da tre elementi principali:

- **Profili (Profiles)**
- **Servizi (Services)**
- **Caratteristiche (Characteristics)**

Lo stack BLE

Generic Attribute Profile (GATT)

- **Profili (Profiles):** è un insieme di servizi che definiscono una specifica funzione. Esistono profili standardizzati, ad es. Heart Rate Profile.
- **Servizi (Services):** Un servizio è un insieme logico di caratteristiche che rappresentano una funzione specifica, e sono identificati da un UUID (Universally Unique Identifier). Gli UUID possono essere:
 - **Standard (16-bit):** UUID definiti dal Bluetooth Special Interest Group (SIG), es. 0x180D per il Heart Rate Service.
 - **Custom (128-bit):** UUID definiti dall'utente per servizi personalizzati.
- **Caratteristiche (Characteristics):** Una caratteristica è l'elemento più piccolo di dati e rappresenta una singola proprietà che può essere letta, scritta o notificata. Ogni caratteristica ha:
 - **UUID:** Un identificatore univoco che descrive la caratteristica.
 - **Valore:** Il dato vero e proprio, ad esempio, una lettura del sensore o uno stato.
 - **Proprietà:** Definiscono il modo in cui la caratteristica può essere accessibile (ad esempio: leggibile, scrivibile, notificabile).
 - **Descrittori (Descriptors):** forniscono informazioni aggiuntive sul dato, come l'unità di misura o la formattazione.

Lo stack BLE

- **ATT** è il protocollo di trasporto dei dati: definisce il meccanismo per leggere, scrivere e notificare attributi tra dispositivi BLE.
- **GATT** organizza gli attributi in una struttura logica (servizi, caratteristiche, profili) e utilizza ATT per trasferire questi dati.

Lo stack BLE



Generic Access Profile (GAP)

definisce come i dispositivi BLE interagiscono tra loro, ossia i meccanismi di accesso, scoperta, connessione e modalità operative dei dispositivi. Mentre il GATT si occupa dello scambio di dati una volta stabilita una connessione, il GAP si concentra sulle modalità di accesso e connessione tra i dispositivi..

Uno sviluppatore deve configurare il dispositivo BLE per essere visibile ad altri dispositivi (in modalità advertising) oppure come scanner in cerca di altri dispositivi. Inoltre, può anche configurare i parametri di connessione (ad esempio, intervalli di connessione per ottimizzare il consumo energetico).

SPBTLE-RF0



Description

The **SPBTLE-RF0** is an easy to use Bluetooth® Low Energy network processor module, compliant with Bluetooth v4.1. The **SPBTLE-RF0** module supports multiple roles simultaneously, and can act at the same time as Bluetooth Low Energy master and slave device.

The entire Bluetooth Low Energy stack and protocols are embedded into the **SPBTLE-RF0** module. The external host application processor, where the application resides, is connected to the **SPBTLE-RF0** module through a standard SPI interface.

The **SPBTLE-RF0** module provides a complete RF platform in a tiny form factor. Radio, antenna and high frequency oscillators are integrated to offer a certified solution to optimize the time to market of the final applications.

The **SPBTLE-RF0** can be powered directly with a pair of AAA batteries or any power source from 1.7 to 3.6 V.

Features

- Bluetooth v4.1 compliant
 - Supports master and slave modes
 - Multiple roles supported simultaneously
- Embedded Bluetooth low energy protocol stack
 - GAP, GATT, SM, L2CAP, LL, RFPHY
- Bluetooth low energy profiles provided separately
- Bluetooth radio performance:
 - Embedded ST BlueNRG-MS
 - Tx power: + 4 dBm
 - Rx sensitivity: - 88 dBm
 - Provides up to 92 dB link budget with excellent link reliability
- Host interface
 - SPI, IRQ, and RESET
 - On-field stack upgrading available via SPI
- AES security co-processor
- Certifications:
 - CE qualified
 - FCC, IC modular approval certified
 - BT SIG qualified
- On-board chip antenna
- Operating supply voltage: from 1.7 to 3.6 V
- Operating temperature range: -40 °C to 85 °C

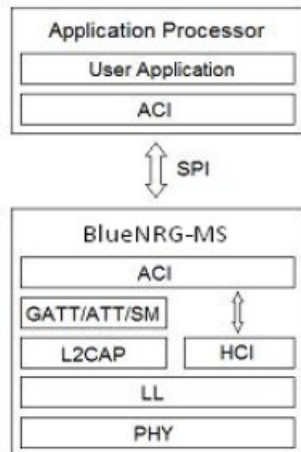
The **SPBTLE-RF0** module embeds the **BlueNRG-MS application processor**. Refer to the BlueNRG-MS web page (<http://www.st.com/en/wireless-connectivity/bluenrg-ms.html>) to access the documentation

ST BlueNRG application processor

Il **BlueNRG-MS** può essere interfacciato con un microcontrollore esterno tramite **SPI**. Il microcontrollore esterno, ad esempio un STM32, può inviare comandi e ricevere dati dal BlueNRG-MS utilizzando un'interfaccia SPI full-duplex.

Il **BlueNRG-MS** include uno stack Bluetooth Low Energy completo che è gestito attraverso comandi e API. La comunicazione tra il tuo microcontrollore e il BlueNRG-MS avviene tramite **HCI (Host Controller Interface)**.

Figure 3. BlueNRG-MS ACI architecture



L'host su cui gira l'applicazione comunica al modulo BLE le informazioni attraverso l' Application command interface (ACI). Sono implementati diversi comandi (documento um1865):

Item	Command	OGF	OCF	OpCode
1	HCI_Disconnect	0x01	0x06	0x0406
2	HCI_Read_Remote_Version_Information	0x01	0x1D	0x041D
3	HCI_Set_Event_Mask	0x03	0x01	0x0C01
4	HCI_Reset	0x03	0x03	0x0C03
5	HCI_Read_Transmit_Power_Level	0x03	0x2D	0x0C2D

⋮

Struttura di un programma

Un programma che utilizzi la periferica BLE è tipicamente costituito dai seguenti passaggi:

1.Inizializzazione del BLE: Usare le librerie fornite dal framework per configurare lo stack BLE. Questo di solito include l'inizializzazione della GAP e della GATT, oltre a definire i ruoli del dispositivo (periferico o centrale).

2.Definizione di Servizi e Caratteristiche: Usando API del BLE stack, definirai i servizi e le caratteristiche personalizzate. Ad esempio, puoi creare un servizio per misurare un parametro ambientale (es. temperatura, umidità).

3.Configurazione dell'Advertising: Configurerai il dispositivo per trasmettere pacchetti di advertising, includendo eventuali dati come il nome del dispositivo o informazioni specifiche dell'applicazione.

4.Gestione degli Eventi BLE: Implementerai il codice che gestisce gli eventi generati dallo stack BLE, come la connessione di un dispositivo esterno o la richiesta di lettura/scrittura di una caratteristica.

5.Ottimizzazione dei Parametri di Connessione: Configurerai gli intervalli di advertising e connessione per bilanciare prestazioni e consumo energetico. Ad esempio, intervalli più lunghi riducono il consumo ma aumentano la latenza.

Steps per la creazione di un progetto partendo da un'applicazione di esempio

I seguenti passaggi fanno riferimento al documento ["STMicroelectronics.X-CUBE-BLE1_GettingStarted.pdf"](#) dove sono descritti più in dettaglio:

1. Creare un nuovo progetto
2. Dalla schermata di configurazione (*.ioc) installare il software pack [X-CUBE-BLE1](#):
 1. Software Packs → Manage Software packs → STM32L4 (solo se non si ha già la repository sul PC)
 2. Software Packs → Select Componets → Installare i pacchetti:
 - X-CUBE-BLE1
 - X-CUBE-BLEMGR
3. Configurare [X-CUBE-BLE1](#) come in figura
4. Tornando al configuratore, configurare i pin delle periferiche e del modulo bluetooth facendo riferimento al documento della board. Alla voce [Middleware and software packs](#) Configurare opportunamente i software packs precedentemente abilitati.
5. Finita la configurazione generare il codice
6. Compilare e caricare il software di esempio così com'è e verificarne la funzionalità tramite l'app di ST disponibile sul playstore

▼ STMicroelectronics.X-CUBE-BLE1	✓	7.0.0	▼
> Exposed APIs	✓		
▼ Wireless BlueNRG-MS	✓	7.0.0	
▼ BlueNRG-MS	✓		
Controller	✓	7.0.0	☒
HCI_TL	✓	7.0.0	Basic ▼
HCI_TL_INTERFACE	✓	7.0.0	UserBoard ▼
Utils	✓	7.0.0	☒
▼ Device BLE1_Applications	✓	7.0.0	
Application	✓	7.0.0	SensorDe... ▼

Steps per la creazione di un progetto partendo da un'applicazione di esempio

1. Guardando il codice notiamo che nell'applicazione di prova i dati trasmessi dal nostro dispositivo sono dei valori casuali e non indicativi del vero stato dei sensori a bordo della scheda
2. Modifichiamo il progetto configurando i sensori come nelle esperienze di laboratorio precedenti copiando i file necessari all'interno del nostro progetto ed includendo i file `"*.h"` dove necessario ed i path alle varie nuove cartelle che abbiamo aggiunto
3. Modifichiamo l'applicazione per leggere e trasmettere i dati dei sensori. È sufficiente modificare la funzione `"User_Process()"` che si trova all'interno di `"app_bluenrg_ms.c"`
4. Se compiliamo vedremo comparire diversi errori che dobbiamo risolvere
5. Rinomiamo l'applicazione come `"TestBLE_Gruppo#"` dove al posto di `#` inserite il numero del vostro gruppo
6. Aprire MATLAB e lo script `"bluetooth.m"`. Modificare opportunamente `NOME_DEVICE` ed eseguire ciascuna sezione del codice una alla volta.